

Ex-no: 6	SCHEDULING ALGORITHMS(FCFS,SJF,PRIORITY,RR)
24/02/2026	

Aim

To study and implement different CPU scheduling algorithms and analyze their performance.

Algorithm

1. Define a set of processes with parameters such as arrival time and burst time.
2. Select a scheduling algorithm (e.g., FCFS, SJF, Round Robin, Priority).
3. Arrange processes according to the chosen scheduling policy.
4. Calculate waiting time and turnaround time for each process.
5. Compute average waiting time and average turnaround time.
6. Draw the Gantt chart to represent process execution.
7. Analyze and compare the results.

CODE:

// FCFS

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    printf("Enter number of processes: \n");
```

```
    scanf("%d", &n);
```

```
    int AT[n], BT[n], CT[n], TAT[n], WT[n];
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
```

```
        scanf("%d %d", &AT[i], &BT[i]);
```

```

}

// Completion Time calculation

CT[0] = AT[0] + BT[0];

for(i = 1; i < n; i++) {
    if(CT[i-1] < AT[i])
        CT[i] = AT[i] + BT[i];
    else
        CT[i] = CT[i-1] + BT[i];
}

float totalTAT = 0, totalWT = 0;

for(i = 0; i < n; i++) {
    TAT[i] = CT[i] - AT[i];
    WT[i] = TAT[i] - BT[i];

    totalTAT += TAT[i];
    totalWT += WT[i];
}

printf("\nAverage Turnaround Time: %.2f\n", totalTAT/n);
printf("Average Waiting Time: %.2f\n", totalWT/n);

return 0;
}

```

```
// SJF
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, j, time = 0, completed = 0, min;
```

```
    printf("Enter number of processes: \n");
```

```
    scanf("%d", &n);
```

```
    int AT[n], BT[n], CT[n], TAT[n], WT[n], visited[n];
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
```

```
        scanf("%d %d", &AT[i], &BT[i]);
```

```
        visited[i] = 0;
```

```
    }
```

```
    while(completed < n) {
```

```
        min = -1;
```

```
        for(i = 0; i < n; i++) {
```

```
            if(AT[i] <= time && visited[i] == 0) {
```

```
                if(min == -1 || BT[i] < BT[min]) {
```

```
                    min = i;
```

```
    }  
    }  
}
```

```
if(min == -1) {
```

```
    time++;
```

```
}
```

```
else {
```

```
    time += BT[min];
```

```
    CT[min] = time;
```

```
    visited[min] = 1;
```

```
    completed++;
```

```
}
```

```
}
```

```
float totalTAT = 0, totalWT = 0;
```

```
for(i = 0; i < n; i++) {
```

```
    TAT[i] = CT[i] - AT[i];
```

```
    WT[i] = TAT[i] - BT[i];
```

```
    totalTAT += TAT[i];
```

```
    totalWT += WT[i];
```

```
}
```

```
printf("\nAverage Turnaround Time = %.2f\n", totalTAT/n);
```

```
printf("Average Waiting Time = %.2f\n", totalWT/n);

return 0;
}

// PRIORITY
```

```
#include <stdio.h>
```

```
int main() {

    int n, i, time = 0, completed = 0, min;

    printf("Enter number of processes: \n");

    scanf("%d", &n);

    int AT[n], BT[n], P[n], CT[n], TAT[n], WT[n], visited[n];

    for(i = 0; i < n; i++) {

        printf("Enter AT, BT and Priority for P%d: \n", i+1);

        scanf("%d %d %d", &AT[i], &BT[i], &P[i]);

        visited[i] = 0;

    }

    while(completed < n) {

        min = -1;
```

```

for(i = 0; i < n; i++) {

    if(AT[i] <= time && visited[i] == 0) {

        if(min == -1 || P[i] < P[min]) {

            min = i;

        }

    }

}

if(min == -1) {

    time++;

}

else {

    time += BT[min];

    CT[min] = time;

    visited[min] = 1;

    completed++;

}

}

float totalTAT = 0, totalWT = 0;

for(i = 0; i < n; i++) {

    TAT[i] = CT[i] - AT[i];

    WT[i] = TAT[i] - BT[i];

    totalTAT += TAT[i];

```

```
    totalWT += WT[i];  
}
```

```
printf("\nAverage Turnaround Time = %.2f\n", totalTAT/n);  
printf("Average Waiting Time = %.2f\n", totalWT/n);
```

```
    return 0;  
}
```

// ROUND ROBIN

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, tq, i, time = 0, completed = 0;
```

```
    printf("Enter number of processes: \n");
```

```
    scanf("%d", &n);
```

```
    printf("Enter Time Quantum: \n");
```

```
    scanf("%d", &tq);
```

```
    int AT[n], BT[n], RT[n], CT[n], TAT[n], WT[n];
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("Enter AT and BT for P%d: \n", i+1);
```

```

scanf("%d %d", &AT[i], &BT[i]);

RT[i] = BT[i];
}

while(completed < n) {

    int done = 1;

    for(i = 0; i < n; i++) {

        if(RT[i] > 0 && AT[i] <= time) {

            done = 0;

            if(RT[i] > tq) {

                time += tq;

                RT[i] -= tq;

            }

            else {

                time += RT[i];

                CT[i] = time;

                RT[i] = 0;

                completed++;

            }

        }

    }

}

if(done)

```



```

        time++;
    }

    float totalTAT = 0, totalWT = 0;

    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];

        totalTAT += TAT[i];
        totalWT += WT[i];
    }

    printf("\nAverage Turnaround Time: %.2f\n", totalTAT/n);
    printf("Average Waiting Time: %.2f\n", totalWT/n);

    return 0;
}

```

OUTPUT:

1. FCFS (First Come First Serve)

```

Average Turnaround Time: 10.33
Average Waiting Time: 5.33

```

2. SJF (Shortest Job First)

```

Average Turnaround Time = 9.00
Average Waiting Time = 4.00

```

3. Priority Scheduling

Average Turnaround Time = 9.67

Average Waiting Time = 4.67

4. Round Robin (Time Quantum = 2)

Average Turnaround Time: 11.33

Average Waiting Time: 6.33

Result

The scheduling algorithms were successfully implemented and analyzed. The waiting time and turnaround time for each process were calculated, and the performance of different scheduling techniques was studied and verified.